

Gradient

Pour un réseau de neurones simple avec un seul neurone, un input x , un output y , une fonction d'activation sigmoïde, et une fonction de perte quadratique, nous devons calculer les dérivées de la perte par rapport au poids w et au biais b .

Notations:

- x : l'input
- y : l'output réel
- $\hat{y}$: l'output prédit par le neurone
- w : le poids
- b : le biais
- $\sigma(z)$: la fonction d'activation sigmoïde, où $z = wx + b$
- L : la fonction de perte quadratique

Fonction Sigmoïde :

La fonction sigmoïde est définie par :

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Fonction de Perte Quadratique :

La fonction de perte quadratique est définie par :

$$L = \frac{1}{2}(\hat{y} - y)^2$$

Dérivée de la Fonction Sigmoïde :

La dérivée de la fonction sigmoïde par rapport à son input z est :

$$\sigma'(z) = \sigma(z)(1 - \sigma(z))$$

Calcul de l'Output Prédit :

L'output prédit $\hat{y}$ est donné par :

$$\hat{y} = \sigma(z) = \sigma(wx + b)$$

Dérivée de la Perte par Rapport à w :

Pour trouver la dérivée de la perte par rapport à w , nous utilisons la règle de la chaîne :

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z} \cdot \frac{\partial z}{\partial w}$$

Calculons chaque terme : 1. $\frac{\partial L}{\partial \hat{y}} = \hat{y} - y$ 2. $\frac{\partial \hat{y}}{\partial z} = \sigma'(z) = \sigma(z)(1 - \sigma(z))$ 3. $\frac{\partial z}{\partial w} = x$

Donc,

$$\frac{\partial L}{\partial w} = (\hat{y} - y) \cdot \sigma(z)(1 - \sigma(z)) \cdot x$$

Dérivée de la Perte par Rapport à b :

Pour trouver la dérivée de la perte par rapport à b , nous utilisons également la règle de la chaîne :

$$\frac{\partial L}{\partial b} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z} \cdot \frac{\partial z}{\partial b}$$

Calculons chaque terme : 1. $\frac{\partial L}{\partial \hat{y}} = \hat{y} - y$ 2. $\frac{\partial \hat{y}}{\partial z} = \sigma'(z) = \sigma(z)(1 - \sigma(z))$ 3. $\frac{\partial z}{\partial b} = 1$

Donc,

$$\frac{\partial L}{\partial b} = (\hat{y} - y) \cdot \sigma(z)(1 - \sigma(z))$$

Résumé des Équations :

- Dérivée de la perte par rapport à w :

$$\frac{\partial L}{\partial w} = (\hat{y} - y) \cdot \sigma(z)(1 - \sigma(z)) \cdot x$$

- Dérivée de la perte par rapport à b :

$$\frac{\partial L}{\partial b} = (\hat{y} - y) \cdot \sigma(z)(1 - \sigma(z))$$

Ces équations peuvent être utilisées pour mettre à jour les paramètres w et b en utilisant la descente de gradient.

```
import numpy as np
```

```
# Paramètres
```

```
x = 3
```

```
alpha = 0.1 # Augmenter le taux d'apprentissage
```

```
target = 10
```

```

# Initialisation des poids
w = 0.5
b = 0.8

# Fonction sigmoïde et sa dérivée
def sigmoid(z):
    return 1 / (1 + np.exp(-z))

def sigmoid_derivative(z):
    return z * (1 - z)

# Boucle d'entraînement
for epoch in range(1000):
    # Forward pass
    net = x * w + b
    out = sigmoid(net)
    loss = (out - target) ** 2

    # Backward pass
    dLoss_dOut = 2 * (out - target)
    dOut_dNet = sigmoid_derivative(out)
    dNet_dW = x
    dNet_dB = 1

    dLoss_dW = dLoss_dOut * dOut_dNet * dNet_dW
    dLoss_dB = dLoss_dOut * dOut_dNet * dNet_dB

    # Mise à jour des poids
    w -= alpha * dLoss_dW
    b -= alpha * dLoss_dB

    # Affichage des résultats
    if epoch % 100 == 0:
        print(f"Epoch {epoch}: w={w}, b={b}, out={out}, loss={loss}")

# Affichage des résultats finaux
print(f"Final: w={w}, b={b}, out={out}, loss={loss}")

```